How Much Observability Does an LLM Agent Need? Leakage-Controlled Root-Cause Analysis over Four Telemetry Modalities

Yuvraj Sehgal
[TODO: affiliation]
Canada
17yuvraj.sehgal@gmail.com

Abstract

LLM agents are rapidly being adopted for root-cause analysis (RCA) in microservice systems, but their reported accuracy is difficult to trust: benchmark artifacts — fault-encoding run identifiers, injection containers named after their faults — hand the agent the answer. We present a leakage-controlled evaluation methodology (identifier masking with cross-tool identity preservation, full-fidelity audit transcripts, and an automated leak auditor) and apply it to a new dataset: 95 labeled fault injections across two architecturally distinct microservice systems, recorded simultaneously in *four* time-aligned telemetry modalities — metrics, logs, distributed traces, and *kernel traces* — with pre-registered fault→modality predictions. We find that naming leaks alone inflate an agent's fault-classification accuracy by 57 percentage points; after closing them, a tool-using agent with generic cross-modality tooling still reaches 83–87% top-1 service localization, roughly double two non-LLM baselines including published state of the art, at ~$0.01 per incident. Degradation experiments yield three counter-intuitive results: accuracy survives 20× trace sampling unchanged; removing kernel telemetry entirely costs little accuracy but 34% more investigation effort; and a *partial* kernel view (raw KPIs without interpretive framing) is *worse than none*. An experience layer of reusable "skills" helps only when skill selection is reliable — mis-selected skills cost more than they contribute. All 500+ diagnoses ship with machine-checkable no-leakage certificates. [TODO: tighten to venue word limit]

CCS Concepts

• **Software and its engineering → Software verification and validation**.

Keywords

root-cause analysis, LLM agents, observability, kernel tracing, benchmark leakage, microservices, AIOps

ACM Reference Format:
Yuvraj Sehgal. 2026. How Much Observability Does an LLM Agent Need? Leakage-Controlled Root-Cause Analysis over Four Telemetry Modalities. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (FSE 2027).* ACM, New York, NY, USA, 7 pages.

1 Introduction

Automated root-cause analysis (RCA) for microservice incidents directly affects time-to-mitigation, and LLM-based agents are the current frontier: they read heterogeneous telemetry, form hypotheses, drive diagnostic tools, and explain their conclusions. But an uncomfortable question precedes any accuracy claim for such agents: *did the agent diagnose the fault, or read the answer off the benchmark?* Injected-fault datasets — including ours, initially — encode their labels everywhere: run identifiers such as `tt_slow_db_aggressive_steady_r1`, injection containers named `noisy-neighbor`, metric fields named `injection`. A language model that sees any of these needs no telemetry at all.

What we did. We built (i) StrataTrace, a two-application, four-modality incident dataset whose kernel-trace layer no existing public dataset provides; (ii) a tool-using LLM RCA agent operating over deterministic, byte-accounted telemetry tools; and (iii) a leakage-control harness — masking, full-fidelity transcripts, and an automated audit — that lets us report, to our knowledge for the first time, *certified hint-free* agent RCA numbers, together with the exact price of the hints we removed.

The evaluation arc is itself a finding. Our agent initially scored 74/74/61 (service / fault-type / both correct). Closing the leaks collapsed it to 48/17/9 — proof that it had been leaning on names. Rebuilding the agent with strictly generic tooling (baseline-vs-incident discipline in every tool, call-graph topology with peer edges, host and limit-proximity channels, fault-taxonomy definitions) recovered 83/48/48 — *above* the leaky number on localization, with every crutch removed. The sequence quantifies both the contamination risk (26 points of service accuracy, 57 points of fault classification) and its recoverability by honest means.

Contributions.

(1) **Methodology.** Leakage-controlled agent evaluation: evidence-preserving identifier masking (with a cross-tool identity requirement — masking must not fragment one entity into several pseudonyms), ground-truth-free audit transcripts, an automated leak auditor, and an A/A variance protocol that bounds single-run claims (±9 points in our setting).
(2) **Dataset and system.** StrataTrace: 95 labeled runs over Sock Shop and Train-Ticket, four time-aligned modalities (~0.001 ms clock skew), a kernel representation ladder (raw CTF → per-second KPIs → wait attribution → natural-language digests), pre-registered fault→modality predictions; plus an open agentic-RCA harness with two non-LLM baselines and a published state-of-the-art adapter.
(3) **Empirical results.** (a) The leak-free agent roughly doubles non-LLM baselines on top-1 service localization; (b) accuracy shows *no cliff* down to 5% trace sampling — traces are ~20×

over-collected for this task; (c) kernel telemetry buys efficiency (−25% agent effort) and induced-DB-latency *typing*, while its accuracy-rescue role is absorbed by good cross-modality tooling; (d) *interpreted* kernel summaries beat raw KPIs — a partial kernel view degrades below kernel-blind; (e) an experience/skill layer is net-negative without reliable evidence-based skill selection; unseen-fault incidents under a full skill library cost ∼18 points (a quantified distractor effect, via leave-one-family-out).

[TODO: one-paragraph roadmap of the paper]

2 Background and Related Work

Non-LLM RCA.. Statistical and spectrum-based localization over metrics and traces is well studied; we compare against a hand-built statistical rule tree and the RCAEval/BARO family [3, 4], whose artifacts we run directly, plus the trace-only methods MicroRank [5] and TraceRCA [2]. **[TODO: verify citations; add spectrum-based and causal-discovery lines]**

LLM and agentic AIOps. Recent systems apply LLMs to incident triage and RCA **[TODO: cite RCACopilot, HolmesGPT, mABC, and 2025–2026 agentic RCA papers]**. To our knowledge none report leakage-controlled numbers; evaluations typically expose dataset-native identifiers to the model.

Benchmark contamination. Label leakage and contamination are recognized threats in ML evaluation **[TODO: cite contamination literature]**; we bring this discipline to agentic AIOps, where the leak channels are structural (identifiers in telemetry) rather than train/test overlap.

Observability datasets. Public microservice incident datasets **[TODO: cite Train-Ticket fault studies [6], RCAEval datasets, Nezha, AIOps challenge corpora]** provide metrics/logs/traces; none include synchronized kernel traces, and none pre-register per-fault modality predictions.

Kernel tracing. LTTng [1] provides low-overhead full-syscall tracing; eBPF-based diagnosis is adjacent **[TODO: cite]**. Our measured collection overhead is −0.5% throughput and +12.6% P95 latency at 200 concurrent users.

3 The StrataTrace Dataset

Subjects. Two systems chosen for architectural contrast. *Sock Shop*: 14 services, one datastore per service, and deliberately partial trace instrumentation (6/14 services emit spans; trace blind spots are a design variable, recorded per fault as `target_trace_visibility`). *Train-Ticket* [6]: ∼40 Java services sharing one MySQL instance — a structurally trace-blind choke point through which every service's queries flow.

Faults. Twelve fault types in five families: host resource exhaustion (CPU, disk, memory, network), per-service resource caps (CPU throttle, memory limit), application faults (induced database latency via an always-in-path proxy; connection error storms), dependency faults (frozen dependency; silent asynchronous queue backlog), and infrastructure faults (noisy neighbor; per-service network degradation). Each recipe writes machine-readable ground truth (target, window, expected blast radius) and an independent verification verdict.

Runs and alignment. Each run is a four-minute session (60 s baseline, 120 s injection, 60 s recovery) with all modalities recorded continuously against one clock; measured cross-modality skew is ∼0.001 ms via per-run (realtime, monotonic, boottime) anchors. 95 canonical labeled runs (46 Sock Shop, 49 Train-Ticket; ∼533 GB compressed, ∼2.7 TB raw); kernel data is 62–64% of every bundle.

The kernel representation ladder.

- **L0**: raw LTTng CTF (full syscalls, scheduler, block, network; 2–13 GB per run) — provenance layer, never read at analysis time.
- **L1**: per-(service, second) KPIs via container PID/TGID attribution (43 services resolved on Train-Ticket despite every container reporting `java`): syscall/block latency percentiles, scheduler, block, network, memory-reclaim rates.
- **L2**: per-service *wait attribution* over the incident window — on-CPU vs. runnable vs. disk vs. off-CPU-external shares — the "why it waited" signal.
- **L3**: templated natural-language deviation digests (baseline-anchored numbers; hallucination-resistant).

Pre-registration. Per-fault winning-modality predictions and scoring rules were frozen before any experiment; deviations discovered during the study are recorded in the catalog's amendment log (§6.3 forces one).

4 Agent, Tools, and Leakage Control

4.1 Agent and tools

A provider-agnostic tool-use loop (GPT-5.4 in all experiments; the harness also drives Anthropic-, Gemini-, and local-model backends) operates over six deterministic, byte-accounted tools: service inventory; server-span latency; *call-graph topology* — span parent/child edges plus *peer edges* derived from terminal client spans, so span-less components (databases, queues, proxies) appear as callees, without which "slow edges converge on the datastore" is structurally unobservable; per-container log *error-rate change* with new-signature detection (chronic signatures are flagged, never decisive); metrics with baseline→incident discipline, a node-level host channel, and *limit-proximity signals* (throttled-seconds, memory-cap evidence — decisive facts invisible to movement-ranked views); kernel evidence (L1 KPI changes, L3 digests, L2 wait attribution); and source-code search over the subject repository. A deterministic *Investigation Context Builder* executes the survey once, records typed claims with provenance in a shared store, and injects a compact evidence brief. The output contract is {service, fault type, evidence, confidence}.

4.2 Leakage control

Masking. The model receives an opaque incident alias (never the run identifier); all fault-vocabulary identifiers (injection containers, proxy names) are deterministically pseudonymized. *Identity preservation matters*: the injected workload appears under different names in different modalities (kernel attribution says `aggressor`; metrics say `anomaly-cpu-stress`); per-string hashing fragments

one entity into several pseudonyms and destroys legitimate cross-modality evidence — worth 26 points of localization in our ablation (§6.2). Real service names are the answer space and remain visible. The agent reasons and answers in alias space; unmasking happens harness-side before scoring.

Transcripts. Every diagnosis records the exact prompts, every raw model response, every tool result both in full and precisely as sent (masked, truncated), per-call usage, and the unmask map. Transcripts are ground-truth-free by construction: the artifact itself certifies that no label reached the model.

Automated audit. A scanner over every model-visible string flags run identifiers, fault tokens under any separator style, injection-container names, and ground-truth vocabulary (static answer-space text — the fault-type taxonomy — is exempted); it exits non-zero on any hit. Every number in this paper is backed by an auditor PASS over its transcripts (500+ diagnoses).

Declared assumptions (not leaks). The exact injection window serves as "alert time" for all methods equally; stress-tool process signatures remain visible (a co-tenant's processes are legitimate SRE evidence); the closed fault-type taxonomy with operational definitions is static answer-space text.

4.3 Variance protocol

Identical-configuration re-runs (A/A) across two full campaigns show ∼20% of per-condition cells flip; condition aggregates carry ±∼9 points. We therefore never quote single-run per-condition deltas inside that band, run r=3 repeats on identified borderline incidents, and prefer per-family flip lists over headline deltas.

5 Experimental Setup

Corpus. 93 labeled incidents (43 Train-Ticket, 50 Sock Shop) form the working set; a 23-incident *gate* (one per fault family per application) bounds agent-cost sweeps. Non-LLM baselines are additionally swept over the full 93 × 15 condition grid.

Methods. (1) a statistical rule tree over the same tools; (2) RCAEval multi-source BARO [3, 4] via its published artifact; (3) MicroRank [5] and TraceRCA [2] (trace-only); (4) the agent. All methods receive identical incident windows; only the agent is subject to masking (which, if anything, disadvantages it — the statistical baseline still exploits container names).

Metrics. Top-1 service (AC@1), fault-type accuracy against a closed 13-label taxonomy, their conjunction (*both*); AC@3/MRR for ranking baselines; per-diagnosis tool calls, bytes touched, tokens (with cache split), and dollar cost.

Degradation conditions. Seeded, deterministic, offline transforms applied before the tool layer; the agent is held fixed within any sweep. Axes: trace retention (100/50/25/10/5% whole-trace sampling), metric resolution (5/10/30/60 s), log level (ALL/WARN/ERROR), kernel tier (all / L1-only / L3-only / none), whole-modality removal, and a kernel×trace interaction grid. All axes, the interaction, and whole-modality removal are reported in §6.4–§6.5.

Model and infrastructure. All agent runs use one hosted reasoning model (Azure OpenAI `gpt-5.4`; no temperature parameter accepted by the model class; max 14 tool-use rounds; transient failures retried with logged backoff). The multi-provider harness (Anthropic/OpenAI-compatible) exists but cross-model replication is future work (§8). Sweeps execute sequentially on a cluster login node at ∼2–2.5 min per diagnosis; a per-axis sweep costs $1–2 at GPT-5-family list prices with 50–73% of input tokens served from the provider prompt cache. **[TODO: pin exact API/model version string for camera-ready]**

Reproducibility. Every result row is keyed by (run, condition spec, seed, method) and linked to its transcript; artifacts ship as sha256-manifested bundles.

Table 1: Leak-free agent vs. non-LLM baselines (top-1 service; both = service and fault type correct).

Method	Service	Both	Degradation behavior
Statistical rule tree	∼48%	38%	flat; kernel-blind
RCAEval/mmbaro	46% (AC@3 63%)	—	flat, 100→5% traces
MicroRank / TraceRCA	∼0% / ∼5%	—	floor, no cliff
Agent (leak-free)	**83–87%**	**57%**	§6.4

Table 2: The honesty arc: identical incidents, progressively honest evaluation.

Configuration	Service	Fault	Both
Leaky (run IDs, fault-named containers visible)	74%	74%	61%
Masked, original tools	48%	17%	9%
Masked, fragmented aliases (identity broken)	26%	17%	4%
Masked, generic tooling rebuilt	**83%**	48%	48%
+ deterministic context brief (config of record)	**87%**	**61%**	**57%**

6 Results

6.1 RQ-A: Honest agent vs. baselines

The leak-free agent roughly doubles both baselines on localization (Table 1) at ∼25k input tokens (50–73% served from prompt cache) and ∼$0.01 per incident. Naive kernel-feature fusion into mmbaro changes nothing (kernel columns rank ∼195th among its change points), and the two non-LLM baselines fail on *different* faults — both observations motivate an agent that reasons across modalities rather than a wider feature table.

6.2 RQ-B: The price of leakage

Naming hints alone were worth ∼26 points of localization and ∼57 points of fault classification — fault labels were effectively read off the run identifier. The fragmented-alias row isolates the identity-preservation requirement of §4.2. Honest tooling then *exceeded* the leaky localization number (Fig. 1).

6.3 RQ-C: What is kernel telemetry worth?

K1. Removing kernel telemetry entirely costs ≤4 points (within the noise band) but +34% investigation effort: with strong cross-modality tooling, kernel's robust value at full telemetry is *efficiency*, not rescue. **K2.** The one stable loss is induced-DB-latency *typing*:

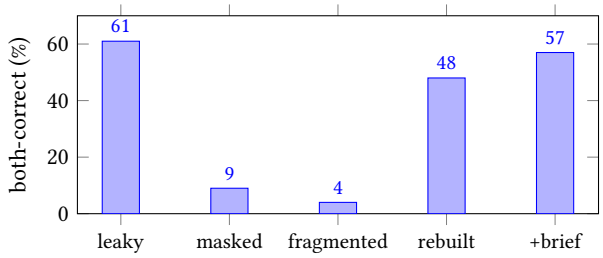

Figure 1: The honesty arc (both-correct on identical incidents): removing leaks collapses performance; rebuilding generic evidence recovers — and on localization exceeds — it.

Table 3: Kernel-tier ablation (23 incidents × 4 tiers; brief on, skills off).

Kernel view	Service	Fault	Both	Agent calls
L1+L2+L3	83%	61%	61%	**9.1**
L1 only	74%	48%	**43%**	10.1
L3 only	78%	57%	57%	10.0
none	78%	57%	57%	12.2

wait attribution converts "the datastore is the locus" into "induced latency, not an application bug". **K3.** *A partial kernel view is worse than none*: raw KPIs without interpretive framing send the agent chasing kernel noise (−14 points vs. kernel-blind; four families lost, none gained) — representation quality dominates kernel presence. **K4.** The pre-registered hypothesis that removing kernel causes the largest drop on blind-spot faults is *not confirmed at full telemetry*: kernel-decisive families hold 85% service in every tier because our limit-signal/topology/host tools absorbed the signal (recorded in the pre-registration amendment log). The interaction test sharpens this (§6.5): the kernel advantage does not grow under trace *thinning* either — but kernel *does* compensate under complete trace *removal*, restoring both locus and typing on induced DB latency. H2 is thereby refined rather than merely refuted: kernel is a compensating channel under modality loss, not under modality thinning. Behaviorally, the kernel-blind agent wastes ~30% of calls re-probing the empty modality unless the tool answers definitively once — an agent-design lesson in its own right.

6.4 RQ-D: Telemetry degradation — no cliff on any axis

Traces. Across 100/50/25/10/5% whole-trace retention (115 diagnoses), both-correct stays within 43–57% (the noise band; 10% retention scores best), 18/23 incidents produce identical outcomes at every level, and the tool mix is constant. Mechanism: the agent consumes spans as *aggregates* — brief latency tops and topology edge percentiles — which survive uniform sampling; direct raw-trace reads are nearly absent. The baselines were flat because they never extracted trace value; the agent is flat because its trace use is statistical and redundantly backed — the same curve for the opposite reason. *Implication:* ~20× *trace collection-cost reduction without diagnostic loss* at otherwise-full telemetry.

Table 4: Degradation axes on the agent (23 incidents per condition; both-correct; single-run noise band ±9 points). Full per-condition tables in the artifact.

Axis	Range tested	Both-correct span	Verdict
Traces	100%→5% retention	43–57%	flat (10% scores best)
Metrics	5 s→60 s scrape	43–52%	flat (30 s scores best)
Logs	ALL→ERROR-only	43–57%	flat (ERROR beats WARN
Kernel	full ladder→none	43–61%	−4 pts, +34% calls; L1-onl

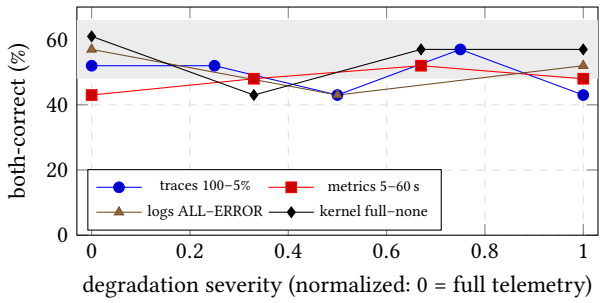

Figure 2: Both-correct under per-axis degradation (23 incidents per point; shaded band = single-run noise around the full-telemetry reference). No axis exhibits a cliff; the kernel dip at severity 0.33 is the L1-only tier (partial view worse than none).

Metrics. Coarsening scrape resolution 5→60 s (92 diagnoses) moves both-correct 43/48/52/48% — non-monotonic, with 30 s scoring best, and per-family flips balanced in both directions (pure churn). The tools compute window-level rates and means over 60–120 s windows; a counter needs only two samples per window, so even 60 s scrape preserves every delta the agent reads.

Logs. Filtering ALL→WARN→ERROR-only (69 diagnoses) gives 57/43/52% — ERROR-only *beats* WARN, confirming non-signal: the log tool keys on error-class signature *changes*, which survive the harshest filter by construction. The one incident recurring in loss lists across both axes (Sock Shop memory-cap) is also in the A/A flip-prone set, so we make no per-family claim.

Consolidated RQ-D answer. There is no degradation cliff at standard operational ranges (Table 4, Fig. 2). The pre-registered cliff expectation resolves as a *robustness* result with one structural cause: the agent consumes every modality at the aggregate level, with evidence redundant across modalities — when one channel thins, the fault signature usually survives in another. The one real hazard is qualitative, not quantitative: a *badly represented* modality (raw kernel KPIs alone, §6.3) hurts more than an absent one. Practically: 5% traces + 60 s metrics + ERROR-only logs cost an agentic-RCA consumer nothing measurable, while kernel telemetry pays via investigation efficiency and induced-DB-latency typing.

6.5 RQ-D′: Interaction and whole-modality removal

Kernel × thinned traces. At 25% and 10% trace retention the kernel-blind agent matches or exceeds the kernel-full agent (both-correct 48 vs. 30% at 25%; 57 vs. 57% at 10%; kernel-decisive families show no kernel edge at any retention), while kernel-blindness costs +1.5–2 calls at every level. The kernel advantage is retention-*invariant* in accuracy and constant in efficiency — there is no compensation effect under thinning. (The 25%-retention cell is weak in every sweep containing it: deterministic seeding keeps the same span sample per run, so one unlucky sample recurs — a seeding artifact we report rather than average away.)

Whole-modality removal. Dropping kernel entirely (78/48/48) replicates the kernel-none tier through a different code path — a free A/A check. Dropping *traces* entirely is the first modality loss that visibly hurts: 70/48/43 (−14 both) at +74% investigation effort, with losses concentrated on the path-shaped Train-Ticket faults whose evidence is topological. The trace no-cliff result is therefore threshold-like: *aggregates survive any sampling; presence matters.* And with traces gone, kernel compensation finally appears in its pre-registered direction: Sock Shop induced-DB-latency stays fully correct on kernel wait-attribution alone. Either modality suffices to localize the datastore; kernel is required for typing; only losing both would break the case. Behaviorally, the no-traces agent spent ∼37% of calls on the two dead tools — the definitive-unavailable answer (§6.3) should generalize to every modality.

6.6 RQ-E: How the investigation itself changes

Every diagnosis carries its full trajectory, giving degradation a behavioral signature to accompany the accuracy curves:

No escalation under thinning. Across trace, metric, and log thinning the tool mix is essentially constant — the agent does not switch strategies because nothing it relies on was lost (its consumption is aggregate-level throughout).

Broad-front widening under loss. When a modality is *removed*, every remaining tool's usage rises together (kernel-blind: logs +67%, topology +32%, metrics +22%; trace-blind: all channels up, +74% total calls) — compensation is diversified rather than a single substitute channel.

Effort is a leading indicator. Tool-call counts rise *before* accuracy falls: kernel removal costs +34% calls at −4 points; trace removal +74% calls at −14. In an operational deployment, investigation effort is an earlier signal of telemetry inadequacy than accuracy, which is only observable with ground truth.

Dead-modality thrashing. Absent a definitive "unavailable" tool answer, the agent re-probes empty modalities per-service: ∼30% of calls under kernel-none, ∼37% under trace-removal went to empty tools. A one-line tool change eliminates the former; we report the latter unfixed for internal consistency of the sweep.

6.7 RQ-F: Does an experience (skills) layer help?

The deterministic Context Builder is the clear win: equal-or-better accuracy at −41% calls and −58% input tokens. Skills lift individual families (co-tenant contention is solved *only* with its skill; cap-fault typing improves) but mis-selection — concentrated on the shared-datastore application, where background DB edges attract db-flavored skills — costs more than skills contribute (selection 15/23). Under leave-one-family-out the selector almost never abstains (1–3/23); distractor skills cost ∼18 points vs. the skill-free floor, though the agent's ability to override a wrong skill caps the damage (19/23 cells unchanged). *Experience layers require selection quality before they are safe on out-of-library incidents.* We release the full selection-confusion data.

Table 5: Skill-library conditions (23 incidents each; masked; selector sees only the evidence digest and may abstain).

Condition	Service	Fault	Both
No skills (floor)	83%	57%	57%
+ context brief (config of record)	**87%**	**61%**	57%
+ full skill library	78%	43%	43%
Leave-one-family-out (unseen fault)	70%	39%	39%

Table 6: Budget configurations (23 incidents each). "MB touched" = telemetry the tools actually consumed per incident; reduction is vs. the full-telemetry configuration of record.

Configuration	Service	Both	Calls	MB touched	Red
Full (config of record)	83–87%	48–57%	9.0	1390	
Lean (5% traces, 60 s, ERROR)	78%	43%	7.6	24.5	5
Minimal (lean − kernel)	83%	39%	11.0	12.1	1

6.8 RQ-G: The minimum observability budget

Each cheap setting is individually free (§6.4); RQ-G asks whether they are free *together*. Largely, yes (Table 6): the lean configuration touches **57× less telemetry** while retaining ≥90% of full localization (78% vs. 83–87%) and both-correct at the lower edge of the single-run band (43% vs. the 48–57% A/A range) — *fault typing, not localization, is the budget-sensitive component.* Removing kernel as well (115×) leaves localization intact (83%) but costs typing further (39%) and raises effort (+45% calls), consistent with kernel's efficiency-and-typing role throughout. Per-incident LLM cost falls to $0.009 at 7.6 calls under lean. The Pareto frontier over all measured configurations is therefore: *minimal* for cheapest localization, *lean* for balanced budget, *full* only when fault typing matters — and since the agent consumes derived kernel views (megabytes), never raw L0, the kernel term of the budget is collection overhead (−0.5% throughput, §3) rather than storage. One incidental observation: Train-Ticket induced-DB-latency, unsolved at full telemetry in this pass, becomes fully correct under lean — thinner traces can *reduce* distraction on borderline incidents (per-family map: Appendix A).

7 Discussion

For benchmark authors. Injected-fault benchmarks leak by construction — through identifiers, not train/test overlap. Masking must preserve cross-tool identity or it destroys legitimate evidence along with the hints. Scores alone are unverifiable; publish transcripts and an executable auditor with them.

For agent builders. Representation beats raw access: interpreted kernel summaries outperform raw KPIs, and a partial view can be worse than none (§6.3). Tools should answer *definitively* when a modality is absent, or the agent thrashes on it. A deterministic context builder bought more accuracy-per-dollar than any prompt refinement we attempted.

For observability practice. At otherwise-full stacks, 5% whole-trace sampling appears safe for LLM-agent RCA — a ∼20× collection saving on the most instrumentation-expensive modality. Kernel tracing costs −0.5% throughput to collect and repays −25% agent effort to use, while uniquely supplying induced-DB-latency typing.

Label semantics vs. mechanism. A closed fault taxonomy penalizes mechanically correct explanations (connection-reset storms described accurately but labeled as outages). We therefore report a *mechanism-adjacent* secondary metric: a conservative, pre-declared four-pair adjacency map (each pair backed by the released evidence audits — e.g., error storm ↔ dependency outage when the agent correctly describes injected connection resets toward the datastore), crediting only diagnoses that already localize the correct service. Under it, the configuration-of-record fault+service score rises from 52% to 65% (n=46, pooled across both campaign passes), and the full-kernel ablation tier from 61% to 70% — a consistent +9–13 point uplift. Label-strict remains the primary score; the adjacency map and all evidence text ship in the artifact so reviewers can apply stricter or looser judgment.

8 Threats to Validity

Internal. Single-run condition aggregates carry a measured ±∼9 point band; we report it explicitly, repeat borderline incidents (r=3), and prefer per-family flip evidence over headline deltas. The injection window doubles as alert time for all methods equally.

External. One LLM (GPT-5.4) — the harness is multi-provider but cross-model replication remains **[TODO: run]**; two subject systems (chosen for architectural contrast); injected rather than organic faults; agent sweeps use the 23-incident gate (93 incidents available) for cost control.

Construct. The closed fault taxonomy penalizes mechanism-correct diagnoses at label boundaries (§7); all degradation axes so far use gentle regimes (uniform whole-trace sampling, scrape coarsening, level filtering) — cliffs may exist under harsher transforms (whole-modality removal, per-span drops, axis interactions). Masking asymmetry favors the *baselines* (the statistical baseline still exploits container names), making our agent-vs-baseline comparison conservative.

9 Conclusion

We asked how much an LLM RCA agent's reported skill survives honest evaluation, and how much observability it actually needs. The answers: naming leaks alone were worth 57 points of fault-classification accuracy; with leaks closed, generic cross-modality tooling still roughly doubles non-LLM baselines on localization at a cent per incident; traces are ∼20× over-collected for this task — degradation by *thinning* costs nothing on any axis, while degradation by *loss* is where modalities earn their keep (removing traces costs 14 points and 74% more effort; kernel then steps in as the compensating channel, alone recovering induced-DB-latency end to end); kernel telemetry is otherwise an efficiency multiplier and the sole source of induced-DB-latency typing, but only in *interpreted* form — raw kernel numbers are worse than none; and experience layers help only as far as their selection is reliable. All claims ship with machine-checkable no-leakage certificates over 600+ transcripts.

[TODO: final data-availability sentence with DOI]

A Per-family results across headline conditions

Table 7 gives the per-family outcome map across the headline conditions (● = service and fault correct, ○ = service only, – = miss). Columns: configuration of record (s0b), kernel tiers (full / L1-only / none), trace removal (noT), and the two budget configurations (lean = 5% traces + 60 s metrics + ERROR logs; min = lean without kernel). Generated directly from the released result rows by the artifact's analysis scripts.

Table 7: Per-family outcomes (S = Sock Shop, T = Train-Ticket).

Family	s0b	kAll	kL1	kNone	noT	lean	min
S: anomaly_cpu	●	●	●	●	●	●	●
S: anomaly_disk	●	●	●	●	●	●	●
S: anomaly_mem	●	●	●	●	●	○	○
S: anomaly_net	–	–	–	–	–	–	–
S: dependency_outage	●	●	●	●	●	●	●
S: error_storm	○	○	○	○	○	○	○
S: noisy_neighbor	○	●	○	●	○	○	●
S: queue_backlog	–	–	–	–	–	○	–
S: slow_db	●	●	●	○	●	●	○
S: svc_cpu_cap	●	●	●	●	●	●	●
S: svc_mem_cap	○	●	○	●	○	–	–
S: svc_net	○	○	–	○	–	○	○
T: anomaly_cpu	●	●	●	●	●	●	●
T: anomaly_disk	●	●	●	●	●	●	●
T: anomaly_mem	●	●	●	●	●	●	●
T: anomaly_net	○	○	○	–	○	○	○
T: dependency_outage	●	●	–	●	–	–	–
T: error_storm	○	○	○	○	○	○	○
T: noisy_neighbor	○	○	○	○	●	●	○
T: slow_db	–	–	–	–	–	●	○
T: svc_cpu_cap	●	●	●	●	–	–	●
T: svc_mem_cap	○	–	○	–	○	○	○
T: svc_net	–	●	–	●	–	–	○

References

[1] Mathieu Desnoyers and Michel R. Dagenais. 2006. The LTTng Tracer: A Low Impact Performance and Behavior Monitor for GNU/Linux. In *Ottawa Linux Symposium*. TODO-verify.

[2] Zeyan Li et al. 2021. Practical Root Cause Localization for Microservice Systems via Trace Analysis. In *IEEE/ACM IWQoS*. TODO-verify authors/venue.

[3] Luan Pham et al. 2025. RCAEval: A Benchmark and Toolkit for Root Cause Analysis of Microservice Systems. TODO-verify: WWW'25 resource track / artifact version 1.6.

[4] Luan Pham, Huong Ha, and Hongyu Zhang. 2024. BARO: Robust Root Cause Analysis for Microservices via Multivariate Bayesian Online Change Point Detection. In *Proc. ACM International Conference on the Foundations of Software Engineering (FSE)*. TODO-verify venue/pages.

[5] Guangba Yu, Pengfei Chen, Hongyang Chen, Zijie Guan, Zicheng Huang, Linxiao Jing, Tianjun Weng, Xinmeng Sun, and Xiaoyun Li. 2021. MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis in Microservice Environments. In *Proc. The Web Conference (WWW)*. TODO-verify pages.

[6] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2018. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Transactions on Software Engineering* (2018). TODO-verify volume/pages (Train-Ticket benchmark).